República Federativa do Brasil

Ministério da Educação

Universidade Federal do Amazonas

Instituto de Computação

Ciência de Dados

Trabalho Prático II: Modelagem de Dados

Professor Dr. Marco Cristo

Alunos: `Nome Sobrenome, Nome Sobrenome e Nome Sobrenome`
E-mail: `nome.sobrenome@icomp.ufam.edu.br, nome.sobrenome@icomp.ufam.edu.br e nome.sobrenome@icomp.ufam.edu.br`

In [1]:
```
# Imports
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats

import sklearn
import sys
import os
```

In [2]:
```
print("Python", sys.version)
print("--------")
print("Pandas:", pd.__version__)
print("Numpy:", np.__version__)
print("SKLearn:", sklearn.__version__)
```

```
Python 3.11.6 (v3.11.6:8b6ee5ba3b, Oct  2 2023, 11:18:21) [Clang 13.0.0 (clang-1300.0.29.30)]
--------
Pandas: 2.2.0
Numpy: 1.26.4
SKLearn: 1.7.2
```

Preliminares: Inicialmente, vamos preparar os dados conforme estabelecido no primeiro trabalho prático. Para tanto, obtenha os dados das cidades dos membros da sua equipe no diretório `data`.

In [3]:
```
DATA_PQT = 'data/enem2019.parquet'
MUNICIPIOS = ['Londrina', 'Manaus', 'Rio de Janeiro'] # Use o que foi especificado para os membros da sua equip
```

In [4]:
```
# Caminho do arquivo
df_enem = pd.read_parquet(DATA_PQT)
df_enem.shape
```

Out[4]: (599982, 56)

In [5]:
```
# Remove todas as instâncias que não possuem valores para as colunas NO_MUNICIPIO_NASCIMENTO ou SG_UF_NASCIMENT
# Para as colunas CO_PROVA_CN, CO_PROVA_CH, CO_PROVA_LC, CO_PROVA_MT e TP_STATUS_REDACAO, remove apenas as inst
# onde todos esses valores encontram-se ausentes.
crit = ((df_enem.CO_PROVA_CN.isnull())&(df_enem.CO_PROVA_CH.isnull())&
        (df_enem.CO_PROVA_LC.isnull())&(df_enem.CO_PROVA_MT.isnull())&
        (df_enem.TP_STATUS_REDACAO.isnull()))
df_enem = df_enem[~crit]
display(df_enem.shape)

notas = [c for c in df_enem.columns if 'NU_NOTA' in c]
for c in notas:
    df_enem[c] = df_enem[c].fillna(0)

cods = [c for c in df_enem.columns if 'CO_PROVA' in c]
for c in cods:
    df_enem[c] = df_enem[c].fillna(500)

# Usando campo NU_IDADE, é criado FAIXA_ETARIA com os seguintes intervalos
# Instâncias com idade inferior a 10 ou superior a 100 anos sao removidas
def to_faixas(idade):
    if idade<18: return '10-18'
```

```
    elif idade<22: return '18-22'
    elif idade<26: return '22-26'
    elif idade<33: return '26-33'
    elif idade<40: return '33-40'
    elif idade<55: return '40-55'
    elif idade<63: return '55-65'
    elif idade<70: return '65-70'
    else: return '70-100'

df_enem = df_enem[(df_enem.NU_IDADE>=10)&(df_enem.NU_IDADE<=100)]
df_enem['FAIXA_ETARIA'] = df_enem.NU_IDADE.apply(to_faixas)

display(df_enem.FAIXA_ETARIA.value_counts())

nota_media = df_enem.NU_NOTA_REDACAO.mean()
df_enem['TARGET'] = df_enem.NU_NOTA_REDACAO.apply(lambda v: 1 if v>nota_media else 0)

cols_to_use = ['FAIXA_ETARIA', 'NO_MUNICIPIO_RESIDENCIA', 'TP_SEXO', 'TP_ESTADO_CIVIL',
               'TP_ST_CONCLUSAO', 'TP_ANO_CONCLUIU', 'TP_ESCOLA', 'CO_PROVA_CH',
               'CO_PROVA_LC', 'NU_NOTA_CH', 'NU_NOTA_LC', 'TP_LINGUA', 'Q001',
               'Q002', 'Q003', 'Q004', 'Q005', 'Q006', 'Q021', 'Q022', 'Q024', 'Q025']

df_enem_c = df_enem[df_enem.NO_MUNICIPIO_PROVA.isin(MUNICIPIOS)][cols_to_use + ['TARGET']].copy()
```

```
(461570, 56)
FAIXA_ETARIA
18-22     193068
10-18     121679
22-26      59240
26-33      42011
33-40      22444
40-55      20065
55-65       2462
65-70        495
70-100        99
Name: count, dtype: int64
```

1 - MODELAGEM: Usando o dataset **df_enem_c**, o seu objetivo é prever que alunos vão obter uma nota na redação acima da nota média observada nas cidades associadas com a sua equipe. Para tanto, prepare um pipeline de comparação justo entre os métodos, onde:

- Variáveis categóricas devem ser codificadas com One-hot;
- Variáveis numéricas devem ser padronizadas (z-score);
- Todos os métodos comparados devem ser otimizados (usando uma ténica de busca em *grid* e validação cruzada) de acordo com o conjunto *mínimo* de hiper-parâmetros descritos na tabela abaixo:

Método	Número mínimo de parâmetros a serem avaliados
Naive Bayes	
Logistic Regression	- **C**: 0.1, 1, 10 - **penalty**: L2 - **solver**: lbfgs, liblinear
Decision Tree	- **max_depth**: None, 8, 16 - **min_samples_leaf**: 1, 5, 18
Support Vector Machine	- **C**: 0.5, 1, 4 - **kernel**: linear, rbf - **gamma**: scale
Neural Network	- **hidden_layer_sizes**: (64,), (128,) - **activation**: relu, tanh - **alpha**: 1e-4, 1e-3 - **learning_rate_init**: 1e-3
Random Forest	- **n_estimators**: 200, 400 - **max_depth**: None, 12 - **min_samples_leaf**: 1, 5
Gradient Boosting	- **learning_rate**: 0.05, 0.1 - **max_depth**: None, 8, 16 - **l2_regularization**: 0.0, 1.0

2 - AVALIAÇÃO COMPARATIVA: Os resultados devem ser comparados usando as métricas **Acurácia**, **F1** e **AUC**. Em particular, todos os métodos devem ser comparados entre si quanto à significância estatística das diferenças entre os AUCs obtidos. Para isso, use o teste não paramétrico de Wilcoxon. Ao fim, as seguintes tabelas devem ser preenchidas:

Método	Acurácia	F1	AUC
Naive Bayes			
Logistic Regression			
Decision Tree			

Método	Acurácia	F1	AUC
SVM			
Neural Networks			
Random Forest			
Gradient Boosting			

Método 1	Método 2	p-value (diferença de AUCs)
Naive Bayes	Logistic Regression	
Naive Bayes	Decision Tree	
Naive Bayes	SVM	
Naive Bayes	Neural Networks	
Naive Bayes	Random Forest	
Naive Bayes	Gradient Boosting	
Logistic Regression	Decision Tree	
Logistic Regression	SVM	
Logistic Regression	Neural Networks	
Logistic Regression	Random Forest	
Logistic Regression	Gradient Boosting	
Decision Tree	SVM	
Decision Tree	Neural Networks	
Decision Tree	Random Forest	
Decision Tree	Gradient Boosting	
SVM	Neural Networks	
SVM	Random Forest	
SVM	Gradient Boosting	
Neural Networks	Random Forest	
Neural Networks	Gradient Boosting	
Random Forest	Gradient Boosting	

3 - EXPLICAÇÃO: Finalmente, reporte a contribuição de cada variável usada pelo **melhor modelo**, usando o SHAP (use summary plots). Ao reportar o nome das variáveis, use nomes interpretáveis (ex: EducacaoDePaiOuResponsavel em lugar de Q001)

In [ ]: